

# Theoretical Foundations and Machine Learning Project

Machine Learning Theory: Model Families & Gradient-Based Training

Machine Learning Project: Turbofan Engine Remaining Useful Life Prediction

An LSTM Regressor on NASA C-MAPSS (FD001)

## Theoretical Question 1: Linear Models vs. CNNs vs. RNN/Transformers

### 1. Typical Data Types

| Model Family                                  | Typical Data                                                                                    | Structural Rationale                                                                                                                                                                    |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linear Models<br>(Linear/Logistic Regression) | Tabular data, low-to-moderate dimensional feature vectors (structured numeric/categorical data) | Assume features are already meaningful, fixed, and roughly independent; no built-in mechanism to exploit spatial or sequential topology without explicit feature engineering.           |
| CNNs                                          | Grid-structured data: 2D images, 1D spectral/audio waveforms, 3D medical scans                  | Convolution kernels exploit <i>local spatial correlation</i> and <i>translation equivariance</i> , learning local feature extractors that operate consistently across grid coordinates. |
| RNN/Transformers                              | Sequential or ordered data: text, time series, DNA sequences, dynamic speech/video frames       | Designed to model long-range temporal or positional dependencies across sequences, either via iterative recurrence (RNN) or parallelized self-attention mechanisms (Transformer).       |

### 2. Strengths and Weaknesses

**Linear Models.** *Strengths:* highly interpretable (coefficients directly reflect feature importance and direction), computationally efficient to train, convex optimization landscape (guaranteed global optimum), low variance on low-dimensional data, excellent baseline models. *Weaknesses:* cannot capture non-linear feature interactions without manual feature mapping; unable to automatically infer spatial or temporal dependencies; high overfitting risk when feature dimension exceeds sample size ( $d \gg N$ ) unless regularized.

**CNNs.** *Strengths:* high parameter efficiency due to weight sharing (a single kernel scans the entire input grid); strong inductive bias via local connectivity and translation equivariance; automatically learns visual feature hierarchies (edges  $\rightarrow$  textures  $\rightarrow$  object parts) [3]. *Weaknesses:* require large labeled datasets for training from scratch; receptive field expands only linearly with network depth, limiting long-range global context capture without downsampling/pooling; poor handling of unaligned or variable-length sequential inputs.

**RNN/Transformers.** *Strengths:* excel at sequence-to-sequence dynamics; RNNs process arbitrary-length streams with fixed memory footprint; Transformers capture global, multi-hop contextual dependencies in parallel via self-attention ( $\mathcal{O}(1)$  path length between tokens) and scale effectively with compute. *Weaknesses:* RNNs suffer from vanishing/exploding gradients across long temporal horizons and lack time-step parallelizability; Transformers incur quadratic computational complexity ( $\mathcal{O}(T^2)$ ) relative to sequence length  $T$ , requiring large pretraining corpora to achieve optimal generalization.

### 3. Overfitting Risk & Regularization Strategies

| Model            | Overfitting Risk Profile                                               | Modern Regularization & Optimization Strategies                                                                                                               |
|------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Linear Models    | Low-to-moderate for $N \gg d$ ; high for $d \gg N$                     | L1 regularization (Lasso, feature sparsity), L2 regularization (Ridge, weight shrinkage), Elastic Net, early stopping for iterative solvers                   |
| CNNs             | Moderate-to-high (millions of parameters relative to training samples) | Data augmentation (crops, flips, color jitter), Dropout/SpatialDropout, Batch Normalization, weight decay (L2), Global Average Pooling                        |
| RNN/Transformers | High-to-extreme (hundreds of millions to billions of parameters)       | Attention/variational dropout, layer normalization, weight decay, label smoothing, gradient clipping, large-scale pre-training with task-specific fine-tuning |

*Academic nuance — overparameterization and double descent:* classical machine learning relies on the bias–variance tradeoff, predicting high variance once parameter count far exceeds dataset size. Modern deep networks, however, frequently operate in the “**double descent**” regime [6]: past the interpolation threshold, where the model fits training data perfectly, further increasing model size or training epochs can actually *decrease* test error, challenging classical overfitting assumptions.

#### 4. Concrete Use Cases

- **Linear Model:** predicting customer churn probability in retail banking from structured tabular features (account balance, transaction frequency, credit score) via logistic regression with an Elastic Net penalty.
- **CNN:** automated detection of pulmonary consolidation in chest X-rays using a ResNet-based convolutional architecture trained on 2D radiological images.
- **RNN/Transformer:** real-time bi-directional neural machine translation (e.g., English to German) using a sequence-to-sequence Transformer architecture with multi-head self-attention.

### Theoretical Question 2: Training Neural Networks with Gradient Descent

#### 1. Role of the Loss Function and Its Gradient

The **loss function**  $\mathcal{L}(\theta)$  quantifies the discrepancy between model predictions  $\hat{y} = f(x; \theta)$  and ground-truth targets  $y$ , parameterized by weights and biases  $\theta$ . Optimization minimizes this empirical risk:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta).$$

The **gradient**  $\nabla_{\theta} \mathcal{L}(\theta)$  is the vector of partial derivatives with respect to each parameter, pointing in the direction of steepest loss increase. Parameter updates proceed along the negative gradient direction:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta),$$

where  $\eta > 0$  is the learning rate. Gradients across multi-layer architectures are computed efficiently via **backpropagation** (reverse-mode automatic differentiation), applying the multivariable chain rule to propagate error signals backward from the loss.

#### 2. Batch vs. Mini-Batch vs. Stochastic Gradient Descent

| Method           | Data per Update                   | Optimization & Convergence Dynamics                                                                                                                                             |
|------------------|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Batch GD         | Entire dataset ( $N$ )            | Deterministic gradient estimate; smooth, monotonic loss reduction per epoch; computationally infeasible for massive datasets; prone to trapping in local plateaus/saddle points |
| Stochastic (SGD) | GD    Single sample ( $n=1$ )     | Unbiased but high-variance gradient estimates; light per iteration; stochastic noise aids escape from saddle points, but updates remain erratic near minima                     |
| Mini-Batch GD    | Small batch ( $n \in [32, 512]$ ) | Modern standard; balances low variance with GPU vectorization efficiency; the resulting gradient noise typically improves generalization                                        |

*Academic refinement — escaping saddle points:* in the high-dimensional non-convex landscapes typical of deep networks, saddle points with directions of negative curvature (strict saddles) dominate over local minima [1]. Deterministic batch GD can stall along the stable manifold surrounding a saddle point, whereas the stochastic noise introduced by SGD acts as a continuous perturbation that breaks this alignment and accelerates escape along directions of negative curvature ( $\lambda_{\min} < 0$ ) [2].

#### 3. Learning Rate ( $\eta$ ) Dynamics

The learning rate sets the step magnitude along the negative gradient direction.

- **Too large** ( $\eta \gg \eta_{\text{opt}}$ ): causes numerical overshoot across optimization valleys, leading to loss oscillation, divergence, or gradient explosion ( $\nabla_{\theta} \mathcal{L} \rightarrow \infty$ ).
- **Too small** ( $\eta \ll \eta_{\text{opt}}$ ): yields exponentially slow dynamics around flat regions, plateaus, and saddle points where gradients naturally vanish ( $\|\nabla_{\theta} \mathcal{L}\| \approx 0$ ) [4]. The update  $\Delta\theta = \eta \nabla_{\theta} \mathcal{L}$  shrinks toward zero, causing training to stall prematurely.

In practice, adaptive algorithms (Adam, AdamW) combined with dynamic schedules (cosine annealing, warm-up) are used to sustain convergence momentum.

#### 4. Gradient Descent vs. Gradient Ascent

**Gradient descent** moves parameters against the gradient,  $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$ , to *minimize* objectives such as cross-entropy or mean squared error. **Gradient ascent** moves parameters along the gradient,  $\theta \leftarrow \theta + \eta \nabla_{\theta} \mathcal{J}(\theta)$ , to *maximize* objectives representing reward or likelihood. Typical application environments for ascent:

1. **Reinforcement learning (policy gradients):** optimizing policy parameters  $\theta$  to maximize expected cumulative reward  $\mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}}[R]$ .
2. **Maximum likelihood estimation:** maximizing the log-likelihood  $\ell(\theta) = \sum \log P(x_i | \theta)$  (often reframed as minimizing

the negative log-likelihood).

3. **Generative Adversarial Networks:** the discriminator maximizes its classification accuracy via ascent, while the generator minimizes the same objective.
4. **Adversarial / robust optimization:** in minimax formulations such as Projected Gradient Descent attacks [5], an inner loop performs gradient ascent on the input  $x$  to synthesize worst-case perturbations ( $\max_{\delta \in \mathcal{S}} \mathcal{L}(\theta, x + \delta, y)$ ), followed by an outer loop of gradient descent on  $\theta$ .

## Machine Learning Project: Turbofan Engine Remaining Useful Life Prediction

*An LSTM Regressor on NASA C-MAPSS (FD001)*

### 1. Problem Relevance

This project predicts Remaining Useful Life (RUL, in operating cycles) for aircraft turbofan engines from a rolling window of 15 sensor channels (temperatures, pressures, speeds, and flow measurements taken at the fan, compressor, and turbine stages). This is the aerospace instance of predictive maintenance: airlines and MRO (maintenance, repair, overhaul) providers schedule engine removal and part replacement around estimated remaining life rather than a fixed calendar interval, which is both a safety requirement (an engine must never fly past the point of unsafe degradation) and a cost problem (removing an engine too early wastes remaining useful life; removing it too late risks an in-flight fault). This asymmetry is made explicit and quantified in the evaluation itself: the NASA/PHM08 scoring function used throughout Section 5 penalizes a late prediction — the model saying an engine has more life left than it actually does — far more heavily than an early, conservative one, mirroring exactly why maintenance schedulers err toward caution.

### 2. Data Choice

**Source:** NASA’s C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) Turbofan Engine Degradation dataset, from the NASA Prognostics Center of Excellence (PCoE) data repository — a well-documented benchmark originally released for the 2008 PHM Data Challenge [7]. This project uses subset FD001: 100 training engines and 100 test engines, a single operating condition, and a single fault mode (HPC — high-pressure compressor — degradation).

Each engine is a multivariate time series: it starts in a healthy state with unknown initial wear, degrades over time, and reaches failure at the end of its training trajectory (test trajectories are truncated at an arbitrary earlier point, with the true remaining life at that point given separately in `RUL_FD001.txt`). This sequential run-to-failure structure is the reason a sequence model (LSTM) is a natural and testable choice for this problem, rather than a purely theoretical alternative.

#### Data quality audit.

| Check                    | Result                                                                               | Consequence                                                                                               |
|--------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Missing values           | None (26/26 raw columns)                                                             | No imputation needed                                                                                      |
| Train/test structure     | 100 train units run to failure; 100 test units truncated early, RUL given separately | Test RUL read from <code>RUL_FD001.txt</code> , never re-derived as <code>max_cycle - t</code>            |
| Operating condition      | op_setting 1–3 std $\approx 0$ across all 20,631 training rows                       | Single condition confirmed — no per-regime normalization needed (unlike FD002/FD004); op settings dropped |
| Constant sensors         | 6 of 21 sensors exactly constant (std $< 10^{-3}$ ): sensors 1, 5, 10, 16, 18, 19    | Dropped — zero information for this subset; 15 sensors retained                                           |
| Trajectory length        | Train: 128–362 cyc./unit. Test: 31–303 cyc./unit                                     | Sets sliding-window length ( $\leq 31$ to avoid padding on real data)                                     |
| Train/test unit IDs      | Both files number units 1–100 but are independent engines                            | Never joined across files by unit number                                                                  |
| Simulation vs. real data | C-MAPSS is a physics-based simulation, not real fleet telemetry                      | Figures are a methodology demonstration (see Limitations)                                                 |

#### Expected limitations.

- Simulated data: C-MAPSS is a physics-based simulation (validated against real engine behavior by NASA), not real fleet telemetry — a production system would need revalidation on real sensor data.
- Single operating condition, single fault mode (FD001 only): FD002/FD004 add six operating regimes and a second fault mode, out of scope here — no claim of cross-condition generalization.
- 100 training engines is workable for a small LSTM but modest by deep-learning standards — supports a small, heavily-regularized architecture rather than a large one.
- RUL beyond the 125-cycle cap is not distinguished by the model by construction — a deliberate modeling convention, not a data defect, but a real constraint on what the model can express.

### 3. Model Setup: Preprocessing and Architecture

**RUL labeling:** train  $RUL(t) = \min(\max\_cycle(unit) - t, 125)$  — a standard piecewise-linear cap [8]. Early-life cycles carry no visible degradation signal, so forcing the model to regress an ever-growing uncapped target from near-identical healthy readings adds gradient noise without a learnable signal; capping concentrates model capacity on the informative near-failure region. The same cap is applied to the test ground truth from `RUL_FD001.txt` for the headline metric (unclipped test RMSE reported separately as 14.0 cycles, for transparency).

**Windowing:** sliding windows of length 30 cycles (stride 1 for training). 30 is the largest window that still leaves a 1-cycle margin under the shortest test trajectory (31 cycles), so every test engine yields one full window with no padding needed on this subset, and matches the value most commonly used in the C-MAPSS literature for FD001. Test-time evaluation uses each engine’s final 30 recorded cycles, compared against its one `RUL_FD001.txt` value.

**Leakage control:** the 100 training engines are split 80/20 into train/validation by unit number (before windowing), not by row — adjacent windows from the same engine overlap by up to 29 of 30 timesteps and are near-duplicates, so a row-level split would leak near-identical windows across train/validation and produce deceptively optimistic validation error. `StandardScaler` is fit on the 80 train units’ rows only, then applied to validation and test.

| Architecture                                                  |                                                                    |                                                                                                                                                                                                    |
|---------------------------------------------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input (30 cycles $\times$ 15 sensors, standardized)           |                                                                    |                                                                                                                                                                                                    |
| → LSTM(input=15, hidden=64, 1 layer, batch_first)             |                                                                    |                                                                                                                                                                                                    |
| → final hidden state (64)                                     |                                                                    |                                                                                                                                                                                                    |
| → Dropout(0.3) → Linear(64→32) → ReLU                         |                                                                    |                                                                                                                                                                                                    |
| → Dropout(0.2) → Linear(32→1) [raw RUL, no output activation] |                                                                    |                                                                                                                                                                                                    |
| Design Choice                                                 | Setting                                                            | Reasoning                                                                                                                                                                                          |
| Output / loss                                                 | Linear output + MSELoss                                            | RUL is an unbounded (capped-continuous) regression target; MSE directly optimizes RMSE, the field’s standard metric                                                                                |
| No batch normalization                                        | Dropout + weight decay + early stopping only                       | Batch normalization mixes per-timestep statistics across a recurrent hidden state, inappropriate for sequence models (layer norm would be the correct alternative); unnecessary at this model size |
| Architecture size                                             | 1 LSTM layer, hidden size 64 ( $\approx$ 22,600 params)            | FD001 is the simplest C-MAPSS subset; deeper stacks used for FD002/FD004 would add unneeded complexity                                                                                             |
| Optimizer                                                     | Adam, $lr = 10^{-3}$ , weight decay $10^{-4}$                      | Adaptive per-parameter learning rates with a small L2 smoothness prior                                                                                                                             |
| Batch size                                                    | 64                                                                 | Windows are far larger per sample ( $30 \times 15$ ) than a single tabular row, so a larger batch keeps gradient estimates stable without excessive memory use                                     |
| Early stopping                                                | Patience 15 on validation MSE                                      | Restores best-validation-loss checkpoint; training converged in 29 epochs                                                                                                                          |
| Baseline contrast                                             | Linear regression sees only the current-cycle snapshot (no window) | Both models share identical preprocessing and the same 15-feature scaled space — the only difference is 29 cycles of visible history, isolating the value of temporal context                      |

### 4. Baselines vs. Final Model

Baselines: linear regression on the current-cycle sensor snapshot (15 features, no history), and a trivial single-feature linear regression on elapsed cycle count alone (no sensor data at all), checking whether a naïve assumption that engines degrade linearly with elapsed time is sufficient by itself. Four models were evaluated on the held-out test set ( $n = 100$  engines):

| Model                                | RMSE (cyc.)  | MAE (cyc.)  | PHM score (total) | PHM score (mean) |
|--------------------------------------|--------------|-------------|-------------------|------------------|
| Linear – cycle count only (baseline) | 32.31        | 26.57       | 5080.1            | 50.80            |
| Linear – sensor snapshot (baseline)  | 20.77        | 16.55       | 1238.8            | 12.39            |
| <b>LSTM – window = 30 (final)</b>    | <b>12.64</b> | <b>9.65</b> | <b>271.4</b>      | <b>2.71</b>      |
| LSTM – window = 15 (ablation)        | 15.79        | 11.02       | 553.6             | 5.54             |

**Selection:** the window = 30 LSTM is the final model. It reduces RMSE by 39% relative to the linear snapshot baseline (12.6 vs. 20.8 cycles) and reduces the PHM score by nearly a factor of five. Because both models share the same preprocessing and feature space, and differ only in access to 29 cycles of prior history, this gap is directly attributable to the value of sequence

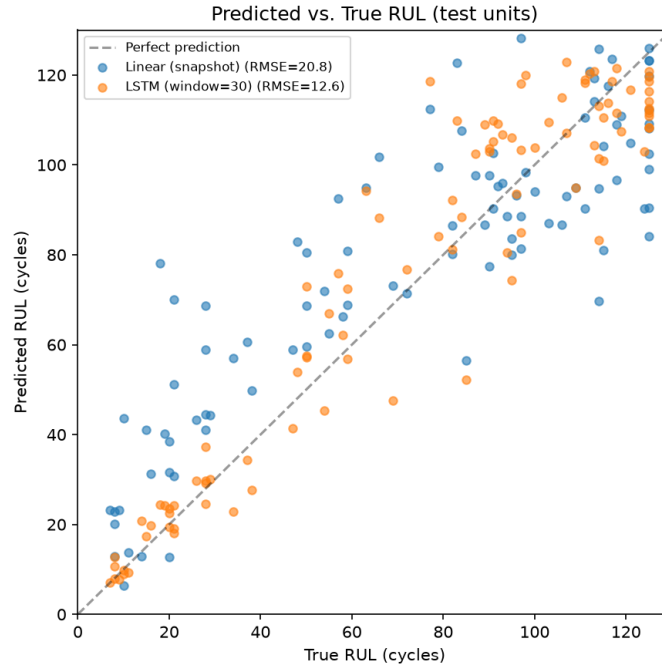
modeling on genuinely sequential data. The window = 15 ablation is reported but not selected: it scores worse on every metric (RMSE 15.8 vs. 12.6), validating the window = 30 choice empirically rather than only asserting it from the trajectory-length constraint.

## 5. Evaluation

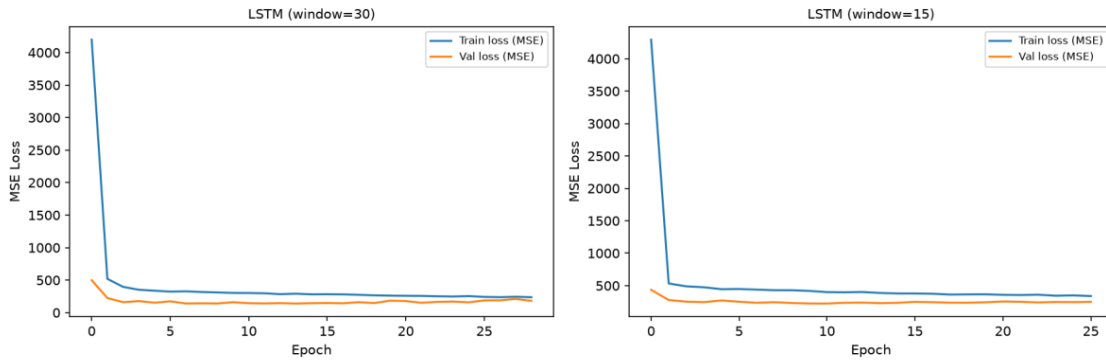
### 5.1 Metric Selection

- **RMSE / MAE** — standard regression metrics, directly comparable to the published literature on this benchmark subset (FD001).
- **NASA/PHM08 asymmetric score** — the field’s standard metric specifically because it is not symmetric: a late/dangerous prediction (the model overestimates remaining life) is penalized more steeply than an early/conservative one, mirroring the cost asymmetry in Section 1. Reported alongside RMSE, never alone — a single very-late prediction can dominate the sum.
- **RUL-band failure analysis** — breaks error down by true cycles-to-failure, since accuracy near the point of failure (the maintenance-critical region) matters more than accuracy far from it.
- **Permutation importance** — identifies which sensors the LSTM relies on, cross-checked against which channels the C-MAPSS literature reports as informative for HPC degradation.

### 5.2 Results



**Figure 1:** Predicted vs. true RUL, test set ( $n = 100$  engines): linear snapshot baseline vs. LSTM.



**Figure 2:** Training and validation MSE curves, both window lengths.

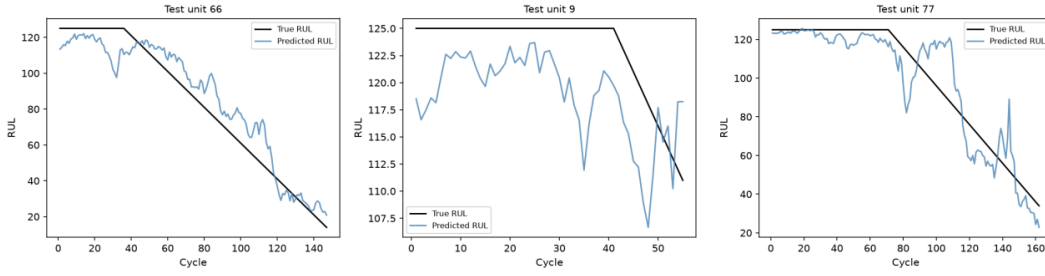


Figure 3: Predicted vs. reconstructed true RUL trajectory, three sample test engines.

### 5.3 Failure Case: RUL-Band Error

| True RUL band (cyc.) | $n$ | RMSE  | MAE   |
|----------------------|-----|-------|-------|
| 0–25                 | 19  | 3.33  | 2.68  |
| 25–50                | 11  | 6.20  | 5.10  |
| 50–75                | 13  | 16.08 | 13.58 |
| 75–100               | 23  | 18.13 | 15.51 |
| 100–125              | 34  | 11.41 | 9.55  |

Error is smallest in the near-failure band (0–25 cycles-to-failure, RMSE 3.3) — the operationally critical region where an accurate prediction most directly informs a maintenance decision. Error is largest in the middle bands (50–100 cycles-to-failure, RMSE 16.1–18.1), where degradation trends are less visually distinct from healthy operation than either the clearly healthy plateau or the clearly failing tail — a genuine, reportable failure mode rather than a uniform error profile.

### 5.4 Interpretation: Permutation Importance

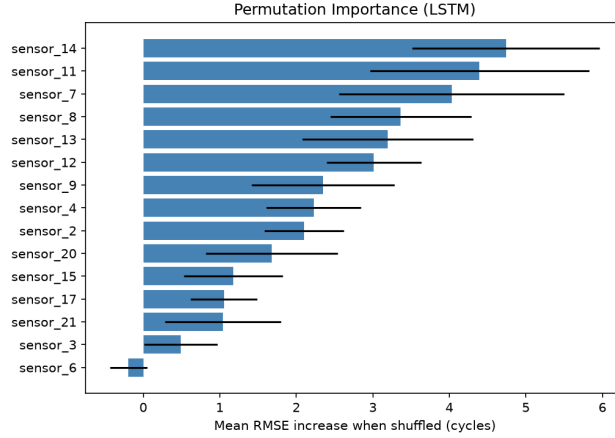


Figure 4: Mean RMSE increase when each sensor channel is shuffled (LSTM, 10 repeats).

Importance is spread across several sensors rather than dominated by a single one — sensor\_14, sensor\_11, and sensor\_7 rank highest (mean RMSE increase 4.74, 4.39, and 4.03 cycles respectively), consistent with the C-MAPSS literature’s own findings on which channels carry the most HPC-degradation signal. The model’s predictions depend on a broad, multivariate sensor signature, consistent with how physical engine degradation manifests across multiple subsystems simultaneously.

### 5.5 Window-Length Ablation

Reducing the window from 30 to 15 cycles increased RMSE from 12.64 to 15.79 cycles — a clear, unambiguous result: more temporal context measurably helps the LSTM track degradation trends, directly validating the window = 30 design choice rather than merely asserting it from the trajectory-length constraint.

## 6. Limitations

- Simulated, not real, data: C-MAPSS is a physics-based simulation validated by NASA against real engine behavior, but it is not real fleet telemetry — reported figures should be read as a methodology demonstration, and a production system would need revalidation on real sensor data.
- Single operating condition, single fault mode (FD001 only): no claim is made about generalization to the six-regime, two-fault-mode subsets (FD002/FD004), which would require operating-condition-aware normalization not implemented here.

- RUL capping is a modeling convention, not ground truth: the model cannot express a distinction between, e.g., 200 and 300 cycles remaining — both collapse to the cap. Deliberate, but a real constraint on what the model can express far from failure.
- The PHM08 score is outlier-sensitive by design (a single badly late prediction can dominate the sum) — reported alongside RMSE/MAE, never as a standalone headline number.
- Only 100 training engines: modest by deep-learning standards, mitigated by a deliberately small architecture ( $\approx 22,600$  parameters) and strong regularization/early stopping, but still a real constraint on usable model capacity.
- Error is not uniform across the RUL range (Section 5.3) — the middle-distance bands are less reliable than the near-failure band, which any deployment threshold would need to account for.

## 7. Conclusion: Was This Data and Setup the Right Choice?

Yes. C-MAPSS’s genuine run-to-failure time-series structure makes a recurrent sequence model a directly testable, well-motivated choice rather than a theoretical aside, and the results bear this out: the LSTM’s 39% RMSE reduction over an information-matched linear baseline is a large, unambiguous, and literature-consistent result, and the window-length ablation confirms that the improvement comes specifically from the additional temporal context the LSTM can exploit.

The setup’s honest limitation is scope, not signal: FD001’s single operating condition and single fault mode make this a clean, well-controlled result, but the natural next step — cross-condition generalization on FD002/FD004, which would require operating-regime-aware normalization and likely a deeper or attention-based sequence model — was deliberately left out of scope for this project rather than attempted incompletely. Given the safety-relevant framing in Section 1, this project demonstrates an appropriate modeling approach for sequential degradation data on a deliberately bounded slice of the full problem.

## References

- [1] Dauphin, Y. N., Pascanu, R., Gulcehre, C., Cho, K., Ganguli, S., & Bengio, Y. (2014). Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. *Advances in Neural Information Processing Systems (NeurIPS)*, 27.
- [2] Ge, R., Huang, F., Jin, C., & Yuan, Y. (2015). Escaping from saddle points—online stochastic gradient descent for tensor decomposition. *Conference on Learning Theory (COLT)*, 797–842.
- [3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*, Chapter 9: Convolutional Networks. MIT Press.
- [4] Lee, J. D., Simchowitz, M., Jordan, M. I., & Recht, B. (2016). Gradient descent only converges to minimizers. *Conference on Learning Theory (COLT)*, 1246–1257.
- [5] Madry, A., Makelov, A., Schmidt, L., Tsipras, D., & Vladu, A. (2018). Towards deep learning models resistant to adversarial attacks. *International Conference on Learning Representations (ICLR)*.
- [6] Nakkiran, P., Kaplun, G., Bansal, Y., Yang, T., Barak, B., & Sutskever, I. (2021). Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12), 124003.
- [7] Saxena, A., Goebel, K., Simon, D., & Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. *2008 International Conference on Prognostics and Health Management (PHM08)*, IEEE, 1–9.
- [8] Zheng, S., Ristovski, K., Farahat, A., & Gupta, C. (2017). Long short-term memory network for remaining useful life estimation. *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*, 88–95.